

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí**Equipe Seliga Nesses Devs****Relatório da Sprint 1**

Todos os itens da rubrica foram avaliados; este documento lista somente itens em que foram encontradas falhas ou inconformidades.

Quando um item não aparece a seguir, não foi registrada falha objetiva para ele na inspeção realizada.

Engenharia de Software II

Item avaliado	Falha encontrada
2.2 Backlog da Sprint 1	O arquivo docs/sprint1/tasks.md define tarefas, RFs/RNFs, prioridade e pontos, mas não informa responsáveis por tarefa nem status individual. O próprio cabeçalho ainda indica "Status: Planejado", enquanto o README principal marca a Sprint 1 como entregue, gerando inconsistência de acompanhamento.
2.3 Definition of Done (DoD)	O DoD exige funcionalidade validada no Docker e item concluído. A aplicação sobe em modo dev com Docker Compose, mas os builds formais falham: frontend em tsc por ignoreDeprecations inválido e backend em tsup/dts por routers sem anotação de tipo exportável. Assim, a entrega não cumpre integralmente um critério técnico mínimo de conclusão.
3.2 Coerência entre documentação e implementação	Há divergência entre documentação/artefatos e implementação de dados: a modelagem visual apresenta uma tabela de junção entre Question e SessionLog, mas o schema Prisma usa inquiry_ids como Json e o fluxo de envio de pergunta não recebe nem grava session_log_id. Com isso, o aceite da Sprint 1 de vincular a pergunta à sessão não fica implementado.
4.1 Estratégia de branches	Na inspeção local foram encontradas apenas develop e as remotas main/develop. Não havia branches feature/*, fix/*, docs/* ou chore/* disponíveis no repositório local/remoto inspecionado, apesar de o padrão documentado exigir uma branch por task.
4.2 Distribuição temporal dos	Há concentração relevante de entregas e merges próximos da review: muitos commits e merges ocorreram em 27/04/2026 e novamente entre 03/05/2026 e 05/05/2026, incluindo chatbot,

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

commits	login, documentação, burndown, caso de uso e ajustes de Docker.
4.3 Participação dos membros	O shortlog mostra participação desequilibrada e fragmentada por aliases. A conta seliganessesdevs tem 51 commits; Gianluca/Duraxxi/Gian somam volume muito superior aos demais; Lucas Cecon tem 9, Guilherme/gui.oliv3 somam 10, Victor/Vitaixs/victor somam 5 e Allan aparece com múltiplos e-mails. Essa fragmentação dificulta aferir colaboração real por integrante e indica baixa uniformidade de contribuição.

Desenvolvimento Web II

Item avaliado	Falha encontrada
2.2 Organização das rotas	O backend em desenvolvimento responde às rotas principais, mas o build de produção do backend falha na geração de declarações por inferência dos routers em auth.routes.ts, chatbot.routes.ts, questions.routes.ts e routes/index.ts. A organização compila para JS, mas não passa no contrato de build configurado no próprio package.json.
3.2 Exposição de dados sensíveis	Há credenciais de seed e exemplos de senha expostos em documentação e seed, como Admin@123 e Secretaria@123. Embora sejam dados de desenvolvimento, a rubrica pede atenção à exposição de senhas/credenciais; a entrega deveria deixar explícito que são credenciais descartáveis ou usar valores demonstrativos sem aparência de credencial real.
4.2 Organização lógica do frontend	O tipo QuestionPayload no frontend usa status "pending" "answered" "archived", enquanto o backend retorna "ABERTA" "RESPONDIDA". Essa divergência de contrato entre camadas reduz a confiabilidade da integração e pode causar erros de tipagem/estado quando a resposta real for usada.
4.3 Estrutura visual	O frontend não passa no build configurado: docker run seliga-frontend pnpm --filter frontend build falhou em tsconfig.app.json por valor inválido de ignoreDeprecations. Assim, a entrega visual roda em dev, mas não está pronta para build reprodutível.

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

Banco de Dados Relacional

Item avaliado	Falha encontrada
1.2 Persistência correta	A pergunta enviada pelo usuário é persistida, mas não é vinculada à sessão de atendimento. O DTO, o payload do frontend e o service de questions não carregam session_log_id, enquanto os critérios da Sprint 1 pedem que a pergunta fique vinculada à sessão que a originou.
2.1 Qualidade dos comandos SQL	O modelo físico não implementa documentos oficiais, chunks indexados e metadados como entidades próprias, apesar de RF02 exigir repositório estruturado com documentos, trechos e metadados. A solução usa apenas evidence_excerpt e evidence_source diretamente em ChatNode, reduzindo a normalização e a rastreabilidade.
2.1 Qualidade dos comandos SQL	Há divergência entre a modelagem documentada e o schema real: o diagrama de banco mostra uma tabela SessionQuestionJunction, mas o schema Prisma não possui essa tabela e armazena inquiry_ids como Json em SessionLog. Isso enfraquece integridade referencial e coerência relacional.
2.2 Otimização inicial	O schema possui índices para parent_id/display_order e status/created_at, mas não há índices para consultas prováveis por slug ativo ou filtros de SessionLog por node_id/flag/data. Como logs e navegação são centrais para a Sprint 1, a ausência desses índices pode prejudicar consultas administrativas futuras.

Técnicas de Programação I

Item avaliado	Falha encontrada
1.1 Aplicação funcional	A aplicação sobe em modo desenvolvimento via Docker Compose e os endpoints health/nodes/root respondem, mas os builds formais falham. Frontend: tsc -b falha com TS5103 em ignoreDeprecations. Backend: tsup falha na etapa DTS por tipos inferidos dos routers. Isso impede considerar a aplicação tecnicamente pronta fora do modo dev.
2.1 Tipagem TypeScript	Há uso de any em código de produção: questions.controller.ts tipa request, response e next como any; pagination.utils.ts recebe page e limit como any. Também há non-null assertion em useChatNavigation.ts. Esses

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

	pontos violam o padrão documentado de evitar any e reduzem robustez.
2.2 Uso de conceitos de POO	O backend usa classes para controllers/services, mas com pouco encapsulamento efetivo: controllers instanciam services diretamente, não há repositories e parte relevante permanece procedural. Para o critério específico de POO, a entrega demonstra uso superficial de classes e baixa exploração de responsabilidades/abstrações orientadas a objeto.